Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-210БВ-25

Студент: Чамара Р.Р.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 22.10.26

Москва, 2026

Постановка задачи

Вариант 8.

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должна создать для решения задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или каналы (pipe). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Родительский процесс создает дочерний процесс. Первой строчкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия файла с таким именем на чтение. Стандартный поток ввода дочернего процесса переопределяется открытым файлом. Дочерний процесс читает команды из стандартного потока ввода. Стандартный поток вывода дочернего процесса перенаправляется в pipe1. Родительский процесс читает из pipe1 и прочитанное выводит в свой стандартный поток вывода. Родительский и дочерний процесс должны быть представлены разными программами.

В файле записаны команды вида: «число число число<endline>». Дочерний процесс производит деление первого числа команды на последующие числа в команде, а результат выводит в стандартный поток вывода. Если происходит деление на 0, то тогда дочерний и родительский процесс завершают свою работу. Проверка деления на 0 должна осуществляться на стороне дочернего процесса. Числа имеют тип int. Количество чисел может быть произвольным.

Общий метод и алгоритм решения

Использованные системные вызовы:

- pid_t fork(void); – создает дочерний процесс, который является копией родительского.
- int execl(const char *path, const char *arg, ...); – заменяет образ текущего процесса программой path (вариант семейства exec, обертка над execve).
- int pipe(int pipefd[2]); – создает неименованный канал: pipefd[0] – конец для чтения, pipefd[1] – конец для записи.
- int dup2(int oldfd, int newfd); – делает дескриптор newfd копией oldfd; используется для перенаправления стандартных потоков ввода и вывода.
- int open(const char *pathname, int flags); – открывает файл и возвращает его дескриптор.
- ssize_t read(int fd, void *buf, size_t count); – читает до count байт из дескриптора fd.
- ssize_t write(int fd, const void *buf, size_t count); – записывает count байт в дескриптор fd.
- int close(int fd); – закрывает файловый дескриптор.
- pid_t waitpid(pid_t pid, int *status, int options); – ожидает завершения дочернего процесса и возвращает информацию о том, как он завершился.
- void exit(int status); – завершает процесс с кодом завершения status.

Программа состоит из двух отдельных исполняемых файлов: parent (родительский процесс) и child (дочерний процесс). Они собираются отдельно с помощью Makefile. Так как использование stdio.h запрещено, весь ввод-вывод выполняется системными вызовами read и write, а вспомогательные функции вывода строк, чисел и сообщений об ошибках (write_all, print_str, print_int, print_error) вынесены в файл utils.c, который подключается к обеим программам.

Алгоритм работы родительского процесса:

- посимвольно читает имя файла со стандартного ввода до символа перевода строки;
- открывает файл на чтение вызовом open;
- создает канал pipe1 и порождает дочерний процесс вызовом fork;
- в дочернем процессе вызовом dup2 перенаправляет стандартный ввод на открытый файл, а стандартный вывод – на записывающий конец pipe1, закрывает лишние дескрипторы и вызовом execl запускает программу child;
- закрывает свои копии дескриптора файла и записывающего конца канала, иначе чтение из канала никогда не получит признак конца данных;
- читает данные из pipe1 и выводит их в свой стандартный поток вывода, пока read не вернет 0;
- ожидает завершения дочернего процесса вызовом waitpid и анализирует код завершения: 0 – успешное завершение, 2 – деление на ноль (родитель выводит сообщение и завершается с тем же кодом), другой код или завершение по сигналу – ошибка.

Алгоритм работы дочернего процесса:

- читает стандартный ввод (на самом деле файл) блоками по 1024 байта и разбирает данные посимвольно, поэтому длина строки и количество чисел в ней не ограничены;
- собирает очередное число из цифр в переменной типа long long и проверяет, что оно помещается в int;
- первое число строки становится результатом, на каждое следующее число результат делится;
- перед каждым делением проверяет делитель: при делении на 0 выводит сообщение в stderr и завершается с кодом 2; также обрабатывается переполнение INT_MIN / (–1);
- в конце строки выводит результат в стандартный поток вывода, который перенаправлен в pipe1; пустые строки пропускаются, некорректные символы приводят к завершению с кодом 3.

О делении на ноль родительский процесс узнает по коду завершения дочернего: после завершения дочернего процесса канал закрывается, родитель получает конец данных, вызывает waitpid, видит код 2 и тоже завершает работу. Результаты всех системных вызовов проверяются; при ошибке выводится сообщение с описанием причины по значению errno.

Код программы

parent.c

```
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>
#include <sys/wait.h>
#include "utils.h"

int main() {
    char filename[256];
    int len = 0;
    char c;

    print_str(STDOUT_FILENO, "Enter file name: ");
    while (1) {
        int r = read(STDIN_FILENO, &c, 1);
        if (r < 0) {
            print_error("parent: read");
```

```
        exit(1);
    }
    if (r == 0 || c == '\n') {
        break;
    }
    if (len >= 255) {
        print_str(STDERR_FILENO, "parent: file name is too long\n");
        exit(1);
    }
    filename[len] = c;
    len++;
}
filename[len] = '\0';

if (len == 0) {
    print_str(STDERR_FILENO, "parent: empty file name\n");
    exit(1);
}

int file = open(filename, O_RDONLY);
if (file < 0) {
    print_error(filename);
    exit(1);
}

int pipe1[2];
if (pipe(pipe1) < 0) {
    print_error("parent: pipe");
    close(file);
    exit(1);
}

pid_t pid = fork();
if (pid < 0) {
    print_error("parent: fork");
    close(file);
    close(pipe1[0]);
    close(pipe1[1]);
    exit(1);
}

if (pid == 0) {
    if (dup2(file, STDIN_FILENO) < 0) {
        print_error("child: dup2 stdin");
        exit(1);
    }
    if (dup2(pipe1[1], STDOUT_FILENO) < 0) {
        print_error("child: dup2 stdout");
        exit(1);
    }
    close(file);
    close(pipe1[0]);
    close(pipe1[1]);

    execl("./child", "child", NULL);
    print_error("child: execl ./child");
    exit(1);
}

close(file);
close(pipe1[1]);

char buf[1024];
int n;
while ((n = read(pipe1[0], buf, sizeof(buf))) > 0) {
```

```
        if (write_all(STDOUT_FILENO, buf, n) < 0) {
            print_error("parent: write");
            break;
        }
    }
    if (n < 0) {
        print_error("parent: read from pipe");
    }
    close(pipe1[0]);

    int status;
    if (waitpid(pid, &status, 0) < 0) {
        print_error("parent: waitpid");
        exit(1);
    }

    if (WIFEXITED(status)) {
        int code = WEXITSTATUS(status);
        if (code == 0) {
            return 0;
        }
        if (code == EXIT_DIV_ZERO) {
            print_str(STDERR_FILENO, "parent: child found division by zero,
exiting\n");
            exit(EXIT_DIV_ZERO);
        }
        print_str(STDERR_FILENO, "parent: child exited with code ");
        print_int(STDERR_FILENO, code);
        print_str(STDERR_FILENO, "\n");
        exit(1);
    }

    if (WIFSIGNALED(status)) {
        print_str(STDERR_FILENO, "parent: child killed by signal ");
        print_int(STDERR_FILENO, WTERMSIG(status));
        print_str(STDERR_FILENO, "\n");
        exit(1);
    }

    return 0;
}
```

child.c

```
#include <unistd.h>
#include <stdlib.h>
#include <limits.h>
#include "utils.h"

int line = 1;
int count = 0;
int result = 0;
int in_num = 0;
int neg = 0;
int digits = 0;
long long cur = 0;

void line_error(const char *msg) {
    print_str(STDERR_FILENO, "child: ");
    print_str(STDERR_FILENO, msg);
    print_str(STDERR_FILENO, " in line ");
    print_int(STDERR_FILENO, line);
    print_str(STDERR_FILENO, "\n");
}
```

```
void end_number() {
    if (!in_num) {
        return;
    }
    if (digits == 0) {
        line_error("bad number");
        exit(EXIT_BAD_INPUT);
    }

    long long value = cur;
    if (neg) {
        value = -cur;
    }
    if (value > INT_MAX || value < INT_MIN) {
        line_error("number does not fit in int");
        exit(EXIT_BAD_INPUT);
    }

    if (count == 0) {
        result = (int)value;
    } else {
        if (value == 0) {
            line_error("division by zero");
            exit(EXIT_DIV_ZERO);
        }
        if (result == INT_MIN && value == -1) {
            line_error("overflow");
            exit(EXIT_BAD_INPUT);
        }
        result = result / (int)value;
    }
    count++;

    in_num = 0;
    neg = 0;
    digits = 0;
    cur = 0;
}

void end_line() {
    end_number();
    if (count > 0) {
        print_int(STDOUT_FILENO, result);
        print_str(STDOUT_FILENO, "\n");
    }
    count = 0;
    line++;
}

void process_char(char c) {
    if (c >= '0' && c <= '9') {
        in_num = 1;
        cur = cur * 10 + (c - '0');
        digits++;
        if (cur > 2147483648LL) {
            line_error("number does not fit in int");
            exit(EXIT_BAD_INPUT);
        }
    } else if (c == '-') {
        if (in_num) {
            line_error("bad number");
            exit(EXIT_BAD_INPUT);
        }
        in_num = 1;
        neg = 1;
```

```
    } else if (c == ' ' || c == '\t' || c == '\r') {
        end_number();
    } else if (c == '\n') {
        end_line();
    } else {
        line_error("unexpected symbol");
        exit(EXIT_BAD_INPUT);
    }
}

int main() {
    char buf[1024];
    int n;

    while ((n = read(STDIN_FILENO, buf, sizeof(buf))) > 0) {
        for (int i = 0; i < n; i++) {
            process_char(buf[i]);
        }
    }
    if (n < 0) {
        print_error("child: read");
        exit(1);
    }

    if (in_num || count > 0) {
        end_line();
    }

    return 0;
}
```

utils.h

```
#ifndef UTILS_H
#define UTILS_H

#define EXIT_DIV_ZERO 2
#define EXIT_BAD_INPUT 3

int write_all(int fd, const char *buf, int n);
int str_len(const char *s);
void print_str(int fd, const char *s);
void print_int(int fd, int x);
void print_error(const char *msg);

#endif
```

utils.c

```
#include <unistd.h>
#include <errno.h>
#include <string.h>
#include "utils.h"

int write_all(int fd, const char *buf, int n) {
    int done = 0;
    while (done < n) {
        int w = write(fd, buf + done, n - done);
        if (w < 0) {
            return -1;
        }
        done += w;
    }
    return 0;
}
```

```
int str_len(const char *s) {
    int n = 0;
    while (s[n] != '\0') {
        n++;
    }
    return n;
}

void print_str(int fd, const char *s) {
    write_all(fd, s, str_len(s));
}

void print_int(int fd, int x) {
    char buf[12];
    char out[12];
    int i = 0;
    int neg = 0;
    long long v = x;

    if (v == 0) {
        print_str(fd, "0");
        return;
    }
    if (v < 0) {
        neg = 1;
        v = -v;
    }
    while (v > 0) {
        buf[i] = v % 10 + '0';
        v = v / 10;
        i++;
    }
    if (neg) {
        buf[i] = '-';
        i++;
    }
    for (int j = 0; j < i; j++) {
        out[j] = buf[i - 1 - j];
    }
    write_all(fd, out, i);
}

void print_error(const char *msg) {
    int err = errno;
    print_str(STDERR_FILENO, msg);
    print_str(STDERR_FILENO, ": ");
    print_str(STDERR_FILENO, strerror(err));
    print_str(STDERR_FILENO, "\n");
}
```

Makefile

```
CC = cc
CFLAGS = -Wall -Wextra

all: parent child

parent: parent.c utils.c utils.h
	$(CC) $(CFLAGS) -o parent parent.c utils.c

child: child.c utils.c utils.h
	$(CC) $(CFLAGS) -o child child.c utils.c

clean:
```

```
rm -f parent child
```

Протокол работы программы

Тестирование:

```
$ make
cc -Wall -Wextra -o parent parent.c utils.c
cc -Wall -Wextra -o child child.c utils.c
$ cat input.txt
100 5 2
7 2
-20 4
2147483647 1
-2147483648 2
42

  9   3
$ cat input_zero.txt
100 5 2
50 0 2
8 4
$ cat input_bad.txt
10 2
5 abc
$ cat input_no_newline.txt
12 3
100 10
$ ./parent
Enter file name: input.txt
10
3
-5
2147483647
-1073741824
42
3
$ echo $?
0
$ ./parent
Enter file name: input_zero.txt
child: division by zero in line 2
10
parent: child found division by zero, exiting
$ echo $?
2
$ ./parent
Enter file name: input_bad.txt
child: unexpected symbol in line 2
5
parent: child exited with code 3
$ echo $?
1
$ ./parent
Enter file name: input_no_newline.txt
4
10
$ echo $?
0
$ ./parent
Enter file name: nofile.txt
nofile.txt: No such file or directory
$ echo $?
1
```

Вывод

В ходе лабораторной работы я на практике освоил создание процессов, запуск отдельной программы в дочернем процессе, перенаправление стандартных потоков, передачу данных между процессами через неименованный канал и получение кода завершения дочернего процесса. Программа корректно обрабатывает деление на ноль, некорректные входные данные и ошибки системных вызовов. Основная сложность была связана с запретом stdio.h: вывод чисел и сообщений пришлось реализовать самостоятельно поверх write. Также важно закрывать неиспользуемые концы канала: иначе родительский процесс не получает признак конца данных и зависает.